

RESEARCH-BACKED PRODUCT STRATEGY

ThreeNative Product Playbook

From AI-generated 3D demos to verified, editable, shippable native games.



Direct the game.
ThreeNative builds, proves, and ships
it.

Executive recommendation

Build the delegation product, not another game editor.

ThreeNative Studio should be the first commercial wedge: an AI-native environment that turns creative direction into a verified, editable, native game. The open-source ThreeNative framework earns trust, supplies the runtime, and distributes the programming model. Studio monetizes managed autonomy, persistent intent, verification, compute, and builds.

Category

AI-native game creation platform, powered by a batteries-included Three.js game framework and a native WebGPU-first runtime.

Beachhead customer

A technically capable **AI-first game creator** who can direct a vision, accepts delegating implementation, and is frustrated by disposable generated demos.

Primary job

Turn a screenshot, clip, or idea into a small polished game that can be played, continued, and shipped without learning a traditional engine or debugging fragile generated infrastructure.

Moat to pursue

Verified delegation: intent memory, reusable game semantics, runtime inspection, automated playtesting, evidence, native builds, and a growing corpus of verified tasks.

Hard decisions

Do

- ✓ Focus on Studio and one narrow family of games.
- ✓ Keep upstream Three.js and raw TypeScript accessible.
- ✓ Optimize for maximum correct delegation.
- ✓ Make meaningful AI changes reviewable and provable.
- ✓ Charge for managed execution, verification, and platform work.
- ✓ Extract abstractions only after repeated use.

Do not

- ✗ Compete with Unity, Unreal, or Godot on editor breadth.
- ✗ Build a proprietary scene model, mandatory ECS, or closed box.
- ✗ Lead with “faster Three.js” as the product story.
- ✗ Launch an open plugin or game marketplace before supply quality.
- ✗ Target AAA or highly manual studio workflows.
- ✗ Let consoles, multiplayer, or photorealism derail PMF.

Central recommendation: prove that creators repeatedly pay for a trustworthy **intent → plan → build → verify → ship** loop before investing in ecosystem layers. Plugins are the second business surface. A “Steam for vibe-coded games” is a later distribution option with explicit quality and cold-start gates.

Contents

1. Method: The Lean Product Playbook applied to ThreeNative
2. Product definition and category boundaries
3. Why now: market evidence and its limits
4. ThreeNative Product-Market Fit Pyramid
5. Target segments, persona, and anti-persona
6. Underserved needs and jobs-to-be-done
7. Alternatives and competitive position
8. Value proposition and product principles
9. UX strategy for directed autonomy
10. Verification as the core product system
11. MVP feature set, exclusions, and prototype tests
12. Metrics, pricing, and product-market fit
13. Supplemental Lean Canvas
14. Open source, plugins, and the future game marketplace
15. Roadmap, 90-day plan, assumptions, and gates
16. Discovery interview guide and research references

How to use this report: every important statement is either a founder decision, an evidence-backed observation, or a hypothesis. The highest-value next work is not additional platform engineering. It is reducing uncertainty about the target customer, repeated use, willingness to pay, and whether visible verification creates trust.

1. The Lean Product Playbook, applied rather than summarized

Dan Olsen's framework separates the **problem space**—target customers and underserved needs—from the **solution space**—value proposition, feature set, and user experience. The six-step Lean Product Process iterates through those layers toward product-market fit.^{[1][2]}



Olsen's Product-Market Fit Pyramid: market layers form the base; product layers sit above them.

The process used in this report is:

1. Determine the target customer.
2. Identify underserved needs.
3. Define the value proposition.
4. Specify the MVP feature set.
5. Create the MVP prototype and user experience.
6. Test the MVP with target customers, one person at a time, and iterate.

Olsen emphasizes that customers are generally better at reacting to concrete solution artifacts than articulating the underlying problem space. He also recommends testing design artifacts before writing a large amount of code.^{[2][3]}

What this report adds

- A synthesis of the founder's product thesis with one strict category correction: **Three.js is a JavaScript 3D library/rendering layer, not a game engine and not a game framework.**
- Current evidence from developer surveys, AI game-creation products, human-AI interaction research, software ecosystems, and creator marketplaces.
- A concrete PMF Pyramid, provisional persona, Kano-style value proposition, MVP candidate, UX architecture, North Star metric, pricing model, Lean Canvas, and ecosystem sequencing.
- A supplemental Lean Canvas. The Lean Canvas comes from Ash Maurya/Leanstack, not Olsen's book; it is used here only to compress business hypotheses into one view.^[4]

EXTERNAL EVIDENCE

Official sources establish directional facts: AI use is high, trust is weak, incumbents are adopting plan-build-test agents, and Three.js has a large software ecosystem.

UNVALIDATED HYPOTHESIS

No desk-research statistic proves that enough creators will pay ThreeNative Studio. Persona, retention, genre focus, delegation tolerance, pricing, and marketplace demand require direct behavioral tests.

Do not mistake a strong thesis for product-market fit. This report is an operating hypothesis designed to make cheap tests possible.

2. A four-layer product, with Studio as the first business



The sequence is bottom-up. Each upper layer is earned only after the layer below demonstrates recurring use.

Layer	What it is	What it is not	Business role
Three.js	Upstream JavaScript 3D library and renderer, including the real scene graph, materials, loaders, and WebGPU renderer.	Not a game engine, game framework, editor, physics system, or gameplay architecture.	Existing AI fluency, ecosystem, and universal escape hatch.
ThreeNative	A batteries-included game framework and native runtime that preserves Three.js while adding portability, game systems, platform plumbing, and verification affordances.	Not "Unity in TypeScript," not a proprietary scene model, and not a mandatory ECS.	Open-source adoption, trust, technical credibility, integrations, and community.
ThreeNative Studio	An AI-first product where the creator supplies direction and constraints; agents plan, implement, run, inspect, test, and package the result.	Not primarily a manual level editor, chat toy, or single-prompt generator.	Subscription, credits, managed compute, persistent project intelligence, and builds.
Ecosystem	Plugins and eventually verified games that agents can discover, use, test, and distribute.	Not an early open marketplace full of unverified output.	Take rate, listing fees, retention, network effects, and distribution.

// Traditional game engines give humans tools to build games. ThreeNative gives AI the tools to build games while the human directs what gets built.

Recommended category statement

For AI-first creators who can articulate a game vision but do not want to manually author every system, ThreeNative Studio is an AI-native game creation environment that turns references and direction into verified, editable, native games. Unlike chat-based demo generators or editor-centric engines, it preserves upstream Three.js and code ownership, uses reusable game primitives, and proves each iteration through runtime inspection, playtesting, performance checks, and platform builds.

Positioning language

Hero

Direct the game. We'll build, test, and ship it.

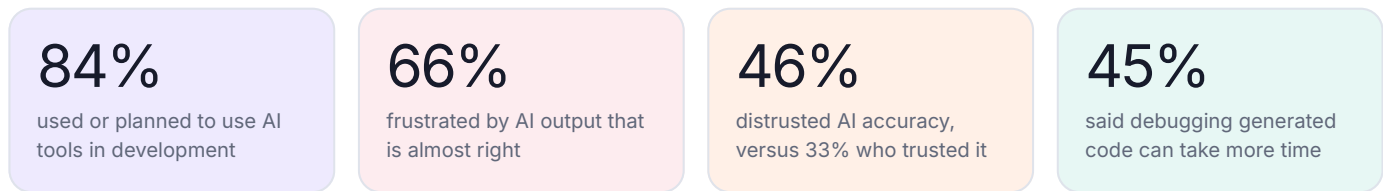
Supporting line

Turn a screenshot, clip, or idea into an editable Three.js game that runs natively on desktop and mobile.

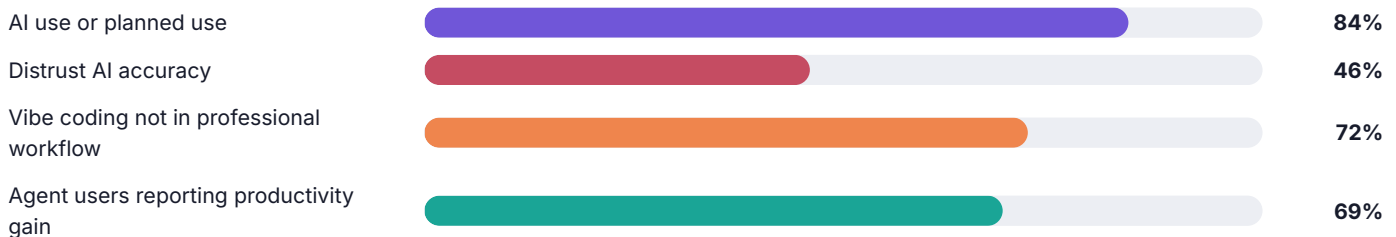
Avoid leading with "AI game engine," "Unity competitor," or "faster Three.js." Those descriptions either misclassify the product or anchor customers to the wrong criteria.

3. Why now: AI adoption is high; confidence is the open wound

The opportunity is not simply that models can write Three.js. The opening is that creators want greater delegation while current AI output remains expensive to trust, debug, and ship.

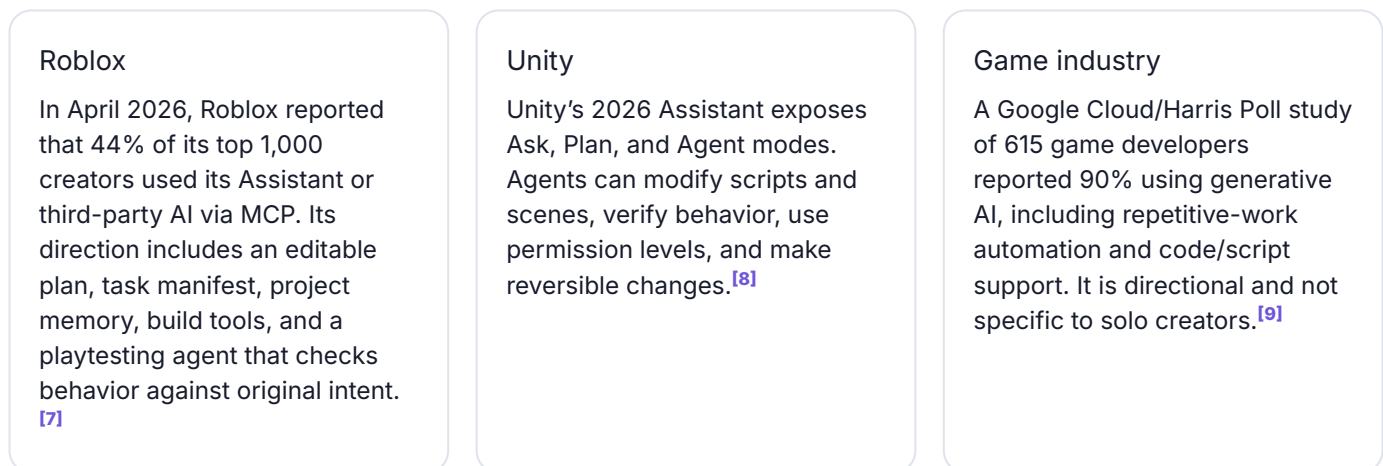


Stack Overflow's 2025 survey used 49,009 responses from 177 countries. Its recruitment channels favor engaged Stack Overflow users, so the results are directional rather than a census.^{[5][6]}



The implication is balanced: the market is real but early. The beachhead is not "all developers." It is creators already comfortable directing AI who feel the cost of verifying and shipping its output.

Game creation is converging on the proposed loop



Three.js provides unusual distribution leverage

The official npm package describes Three.js as a JavaScript 3D library and, at the August 2026 snapshot, showed approximately **13.8 million weekly downloads and 5,397 dependent packages**. Installs are not unique creators, but they signal a broad code corpus, integrations, examples, and model familiarity that a new proprietary API would not possess.^[10]

Strategic conclusion: “AI can generate games” is becoming table stakes. ThreeNative must win on the harder promise that the creator can delegate meaningful work and receive a result that is durable, inspectable, correct enough to continue, and ready for native distribution.

4. The initial ThreeNative Product-Market Fit Pyramid



Layer	Current hypothesis	Critical evidence required
Target customer	Technical AI-first creators who value outcome over manual implementation control and have felt the limits of generated Three.js/browser games.	20–30 interviews, past projects, delegated-work behavior, recruitment conversion, and a clear high-expectation segment.
Underserved needs	Confidence, fast visual/gameplay iteration, durable architecture, native packaging, predictable cost, and freedom to drop down to Three.js.	Importance/satisfaction ranking, current workarounds, time/money lost, and willingness to pay.
Value proposition	“Direct the game. ThreeNative builds, proves, and ships it.”	Message tests, paid concierge work, and comparisons with generic agents and browser-first generators.
MVP	A narrow small-3D-game workflow producing repeated verified checkpoints and at least one reliable native build.	Time to first verified playable, completion rate, manual intervention, and four-week retention.
UX	Reference + intent contract + editable plan + autonomous execution + proof card + rollback + raw code.	One-person moderated tests, correct mental model, safe autonomy, correction, and recovery after failure.

Highest-risk layer: whether the target customer returns after the novelty of the first generated scene. The key signal is repeated delegation across a multi-session project, not “that looks cool.”

5. Beachhead: the AI-first game director

PROVISIONAL PERSONA

Alex, the AI Game Director

Profile: web developer, technical designer, indie hacker, or AI power user. Can read code and articulate gameplay, but does not want to spend years mastering a conventional engine or manually authoring every object.

Mindset: “I own the vision and quality bar. The AI can own most implementation decisions unless I lock something down.”

Evidence of fit: has already produced a Three.js or AI-assisted 3D prototype, then hit brittleness, performance, architecture, testing, or packaging pain.

Success in Alex’s words

- “I gave it a screenshot and direction; it made something recognizably mine.”
- “I asked for faster combat; it changed the game, played it, and showed me what passed.”
- “The project did not collapse after ten prompts.”
- “I shipped Windows and Android without becoming a platform engineer.”
- “When the framework lacked something, I used normal Three.js and TypeScript.”

Segment priority

Segment	Need	Reach	Pay	Recommendation
AI-first technical solo creator	High	High	Medium	Primary beachhead
Web developer entering games	High	High	Medium	Primary adjacent
Technical artist / designer	High	Medium	Medium	Later, after UX proves accessible
Small AI-native team	High	Medium	High	Pro tier after solo PMF
Traditional game developer	Medium	Medium	Medium	Secondary / POC use, not positioning anchor
Established or AAA studio	Low for initial product	Low	High	Anti-persona
Fully nontechnical one-prompt consumer	Unknown	High	Low	Do not target first

The behavioral qualifier matters most

The ideal user accepts handing over substantial implementation control as long as they retain direction, constraints, visibility, and the right to correct or override.

That willingness separates the persona from editor-centric creators even when both are skilled programmers.

Anti-persona

- A 300-person production with specialized level designers, technical artists, custom pipelines, and departmental ownership over every authoring detail.
- A creator whose core satisfaction comes from manually placing, tuning, and wiring every object through a visual editor.
- A team that needs mature console certification, cinematic pipelines, or a decade of marketplace tooling today.

The product should not apologize for this boundary. Avoiding **editor envy** is how ThreeNative preserves focus.

6. The job is not generation; it is trustworthy continuation

Primary job-to-be-done:

“When I have a game idea or reference, help me turn it into a polished, playable, native build without learning a traditional engine or debugging a fragile pile of generated code.”

Underserved-needs hypothesis

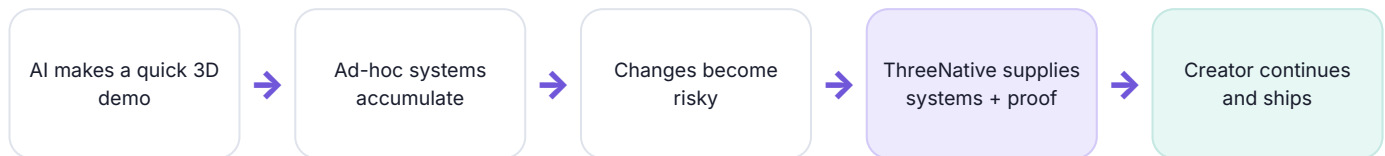
Rank	Need	Why current options underserve it	Observable outcome
1	Confidence that AI output works	Generic agents can compile code yet miss gameplay, visual, physics, performance, and platform regressions.	Every accepted change includes executable evidence and passes agreed checks.
2	Fast direction-to-playable iteration	Prompt generators create a first impression quickly but often lose intent over many edits.	Reference and intent persist; later changes preserve non-negotiables.
3	A durable project rather than a disposable demo	Generated infrastructure is reinvented, inconsistent, and expensive to maintain.	Reusable game semantics, stable architecture, checkpoints, and successful multi-session work.
4	Native performance and packaging without platform expertise	Browser/WebView wrappers and platform-specific build stacks add friction, especially on mobile.	Supported native builds succeed predictably and meet measured performance budgets.
5	Useful delegation without loss of agency	Chat-only tools hide changes; manual engines require too much operation.	Editable plan, locks, permissions, activity trace, rollback, and raw code.
6	Lower token and defect cost	Agents repeatedly write physics, input, ragdoll, state, and asset plumbing from scratch.	Verified primitives shrink generated code while remaining overridable.
7	A path to publish and monetize	Prototype tools often end at a browser link; distribution remains separate.	Export to existing stores first; curated ecosystem and discovery later.

Push and pull

- Excitement from ideas becoming playable quickly.
- Frustration with games that become brittle or slow.
- Desire to ship on mobile/desktop without wrapper and build-stack pain.
- Limited time to master a conventional editor and engine ecosystem.

Anxieties and habits

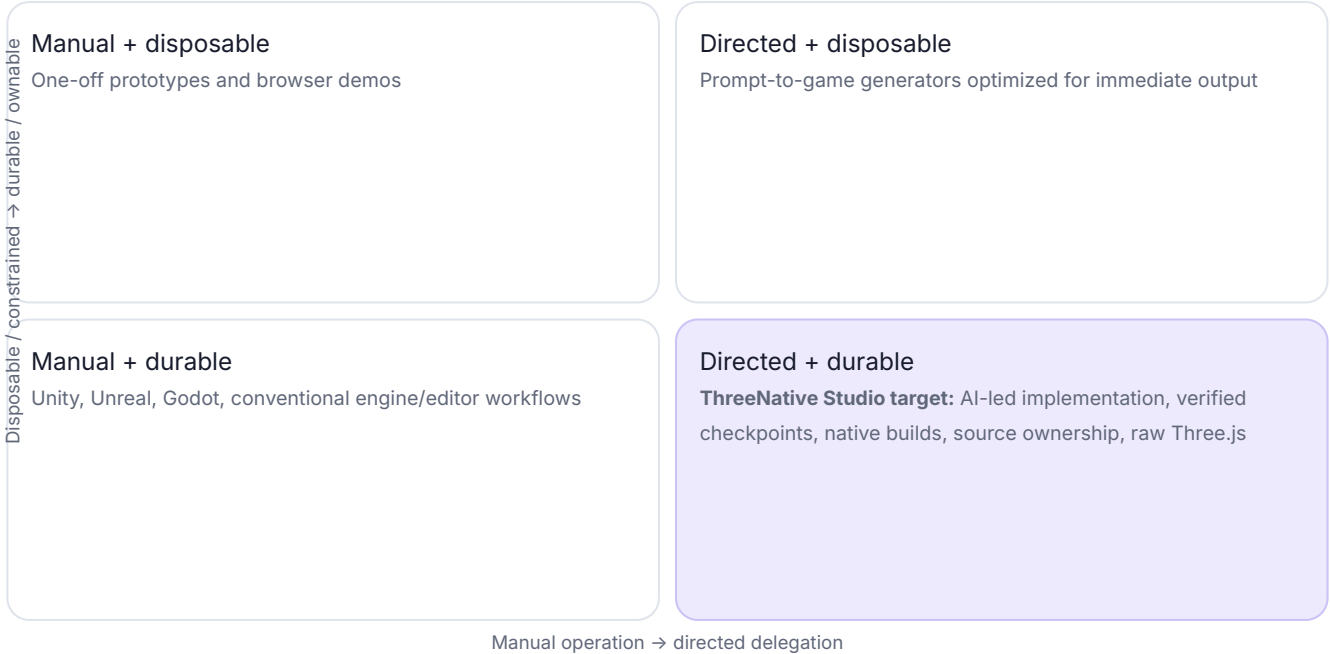
- Fear the agent will silently destroy working behavior.
- Fear credits will disappear without a usable result.
- Fear generated games will look generic or use risky assets.
- Habit of using ChatGPT/Codex plus local Three.js because it feels controllable.



Interview warning: do not ask "Would you use an AI game builder?" Ask for the last game they tried to make, where it broke, what they did next, how much time they lost, whether they abandoned it, and what they actually paid for.

7. Competitive position: directed and durable

ThreeNative does not need to beat conventional engines on their own axis. It needs to own an under-served quadrant: direction-first creation with durable code, open escape hatches, native portability, and built-in verification.



Alternative	Strength	Gap / implication
Generic coding agents	Excellent code generation, repo access, broad model choice.	No default game semantics, runtime state, playtesting harness, visual proof, or managed platform packaging. Integrate with them; models are not the moat.
Rosebud	Plain-language browser creation, Three.js-based 3D, editable JavaScript, assets, Windows export, one-click publishing. ^[11]	Validates demand. Differentiate through native-first execution, multi-session durability, deeper proof, and an open framework.
GDevelop	Open source, visual/no-code workflow, 130+ behaviors/extensions, JavaScript escape hatch, AI, broad exports. ^[12]	Proves “batteries + extension + export.” Do not copy its editor; outperform on AI-native delegation and Three.js fluency.
Roblox Studio	Integrated creation, plan/build/test agent, playtesting, distribution, creator economy. ^[7]	Strong validation and threat, but bound to Roblox. ThreeNative's opening is code ownership and independent native distribution.
Unity + AI	Mature engine/editor, platform reach, project-aware agent, permissions, reversibility. ^[8]	Incumbents will absorb basic AI assistance. ThreeNative must have lower conceptual overhead and a direction-first UX.
Traditional engines	Maximum manual control, mature pipelines, proven large-team production.	Correct answer for the anti-persona. Do not chase parity.

Better
Faster path from creative intent to a trustworthy playable checkpoint; less platform and infrastructure work.

Different
AI is the primary operator; the human directs, locks, judges, and overrides. The framework remains open underneath.

Defensible
Game-specific verification, intent history, reusable semantics, platform conformance, and task/evaluation data compound with use.

Competitive warning: Roblox and Unity already demonstrate editable plans, permissions, reversible work, and automated testing. These are table stakes for Studio, not sufficient differentiation.

8. Value proposition: proof over promises

Olsen recommends choosing which needs to serve and classifying value as **must-haves, performance benefits, and delighters** in the context of alternatives.^[13]

Class	ThreeNative Studio value	Failure mode
Must-have	Playable preview; persistent project and code; reference input; reliable checkpoints; visible changes; raw Three.js/TypeScript; at least one native export; basic asset and game-loop support.	Without these, Studio is a novelty generator.
Performance benefit	Time to verified playable, percentage of requests completed without manual code work, native build success, retained intent, runtime performance, and cost per accepted change.	These determine whether creators continue delegating.
Delighter	The agent plays the game, compares output with the reference, catches its own regressions, auto-iterates within budget, and presents an evidence reel/report.	Delight becomes distrust if verification is theatrical or false.

Ten product principles

- 1. Direction over operation.**
The human chooses intent and quality; AI handles implementation detail.

3. Outcome over implementation.
Default to solving the game problem, not exposing engine ceremony.

5. Native by default.
Web is a target, not the runtime container.
- 2. Proof over promises.**
“Done” means executable evidence, not a confident chat response.

4. Excellent defaults, raw Three.js escape hatch.
Abstractions accelerate; they never imprison.

6. Progressive autonomy.
Trust is earned through plan, permissions, observability, and reversibility.

7. Intent persists.

Visual direction and non-negotiables outlive individual prompts.

9. The creator owns the project.

Portable code, assets, history, and exports.

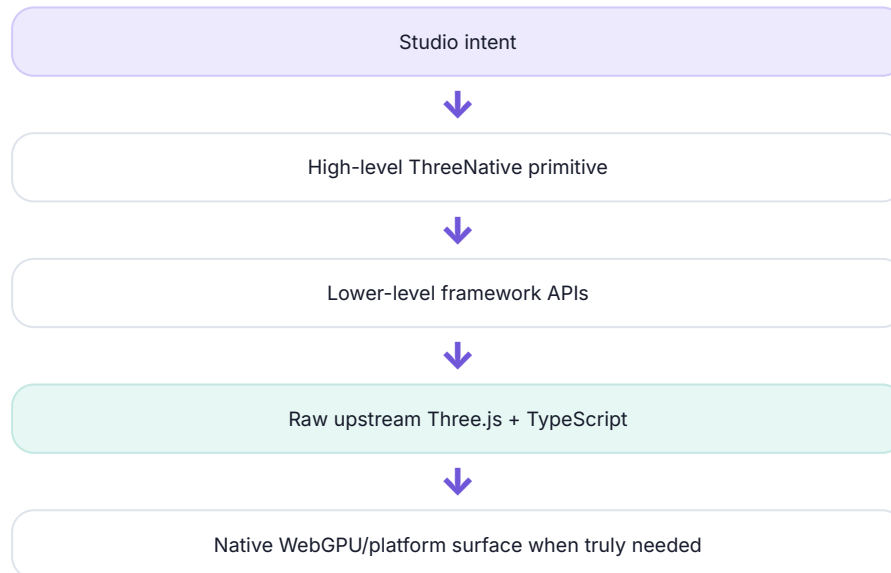
8. Abstractions are earned by repetition.

Extract proven systems; do not speculate a giant engine API.

10. No editor envy.

Build interfaces for direction and verification, not manual parity.

Escape-hatch ladder



The happy path stays concise and AI-legible; the ceiling stays open. Users can introduce their own ECS, physics architecture, editor, map builder, shaders, or custom game systems.

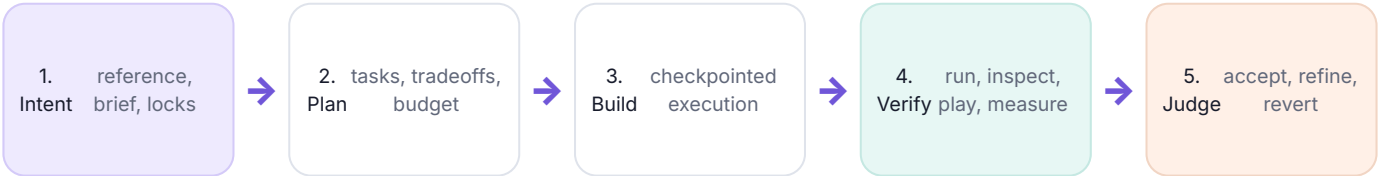
Direct the game. ThreeNative builds, tests, and ships it.

Start from a screenshot, clip, or idea. Get an editable Three.js project, verified playable checkpoints, and native desktop/mobile builds—without living inside a traditional game editor.

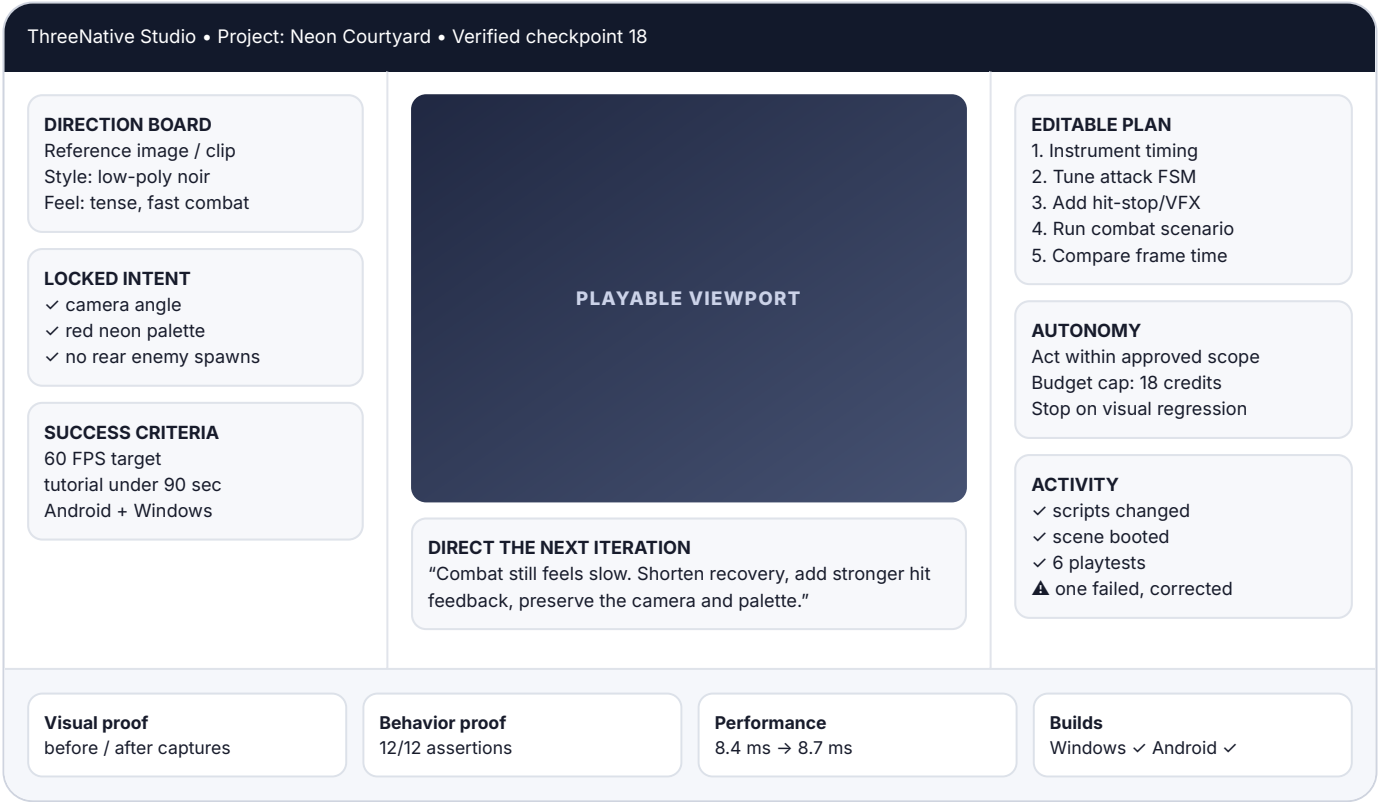
9. UX strategy: design for directed autonomy, not a chat box

Strong AI UX must establish capabilities and limitations, make actions observable, support correction, preserve user control, and recover gracefully. Microsoft’s 18 human-AI interaction guidelines and Google PAIR emphasize these needs.^{[14][15]}

The core loop



Proposed Studio workspace



The intent contract

Before substantial work, Studio translates informal direction into an editable contract:

Vision

What the game should look and feel like, supported by screenshots, clips, references, and language.

Non-negotiables

Objects, mechanics, camera rules, performance floors, brand constraints, and areas the agent must not change.

Acceptance criteria

Observable behavior, visual comparisons, target devices, performance budgets, and a definition of “done.”

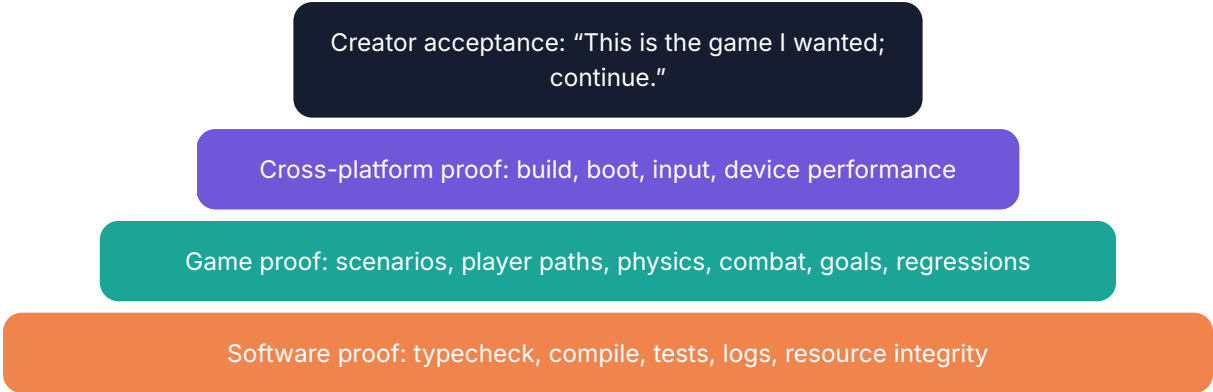
Progressive autonomy ladder

Mode	Agent authority	Appropriate use
Inspect	Read project, scene, logs, performance, and history; no mutation.	Diagnosis, onboarding, trust building.
Plan	Produce editable tasks, expected systems/files, tests, cost, and risk.	Complex or high-impact changes.
Act in scope	Modify approved surfaces and create checkpoints; stop at boundaries.	Normal iteration.
Verify	Run tests, inspect state, control the player, capture evidence, and report failures.	Every meaningful change.
Auto-iterate	Attempt fixes within time/credit limits and quality guardrails.	Trusted users and well-defined work.

Critical UX rule: never create a dead end when AI fails. Preserve the last good checkpoint, explain what was attempted, show the evidence, offer a narrower retry or raw-code path, and do not charge for infrastructure failure.

10. Verification is the core product system

Code generation will commoditize. The durable advantage is knowing what “works” means for a game, collecting the right evidence, and allowing an agent to iterate safely against it.



What a proof card contains

Evidence	Example	Why it matters
Executable checkpoint	Immutable build + source revision + asset hashes.	Reproducibility and rollback.
Visual evidence	Reference, before/after screenshots, image difference, camera metadata.	Appearance and intent drift.
Behavior evidence	Input script, world-state assertions, captured path, success/failure trace.	Proves playability rather than mere rendering.
Runtime inspection	Scene objects, collisions, physics contacts, animation state, audio, errors.	Gives the agent semantic feedback, not pixels alone.
Performance evidence	Frame times, memory, draw/dispatch counts, stutters, device comparison.	Prevents beautiful but unusable output.
Change narrative	What changed, why, systems touched, and known limitations.	Human review and continued ownership.

Verification primitives are framework features

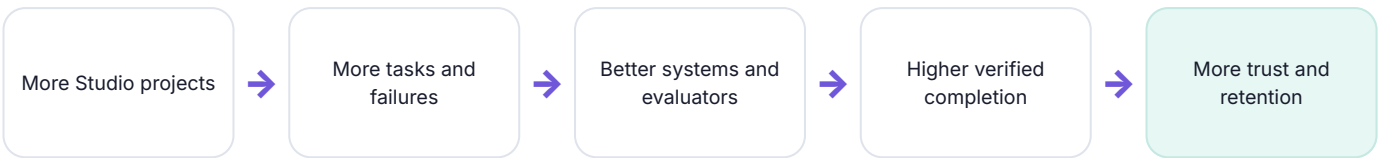
Agent-facing APIs

- Deterministic clock and random seed.
- Synthetic keyboard/controller/touch input.
- Scene and component queries.
- Physics/world snapshots.
- Framebuffer, depth, and object-ID capture.
- Performance counters and budgets.

Reusable assertions

- Player can reach the exit.
- Enemy can detect and damage the player.
- Door opens after interaction.
- No object begins inside a collider.
- Frame time remains below target.
- Reference composition stays within tolerance.

Moat model



Use project data only with explicit and understandable consent. The moat can be built from aggregate failure taxonomy, synthetic tasks, opt-in examples, and open-source conformance—not covert training on customer IP.

AI supplies evidence; it does not certify itself. Automated verification narrows risk, while the creator remains the final judge of fun, taste, and intent.

11. MVP: prove one repeatable vertical slice

Olsen defines the MVP candidate as the minimum functionality required to validate that the product direction is right. It should be tested through prototypes and target-customer feedback before broad implementation.^{[16][17][18]}

MVP promise

From visual reference and brief to a small, verified, editable 3D action/exploration game—with one reliable desktop lane and one reliable mobile lane.

Platform choice should follow current runtime evidence. One excellent desktop and mobile path are more valuable than a broad unreliable matrix.

Included

- ✓ Reference image/clip + brief.
 - ✓ Editable intent contract and success criteria.
 - ✓ Starter archetype: one player, one level, one goal.
 - ✓ Small set of enemies, collectibles, and interactions.
 - ✓ Actual upstream Three.js/TypeScript project.
 - ✓ ThreeNative physics, input, audio, assets, and game loop.
- ✓ Editable agent plan and cost estimate.
 - ✓ Checkpointed changes and rollback.
 - ✓ Scene/log/physics inspection.
 - ✓ Automated smoke, play, visual, and performance checks.
 - ✓ Proof card with captures and traces.
 - ✓ Managed builds for two supported targets.

Explicitly excluded

- Editor parity

No complete gizmo, inspector, terrain, or animation-graph suite. Add targeted controls only when they improve direction or correction.
- Genre breadth

No multiplayer, live service, giant open world, cinematic AAA, vehicle simulation, or “any game from one prompt.”
- Ecosystem breadth

No open plugin store, player marketplace, console productization, or generalized creator economy in the MVP.

Prototype sequence

Test	Artifact	Question	Pass signal
Message test	Three landing pages / short demos.	Which promise creates qualified interest: native Three.js, AI plan-build-test, or screenshot-to-verified-game?	Higher signup and interview completion among target users; users repeat the verification/shipping value in their own words.

Test	Artifact	Question	Pass signal
Concierge / Wizard of Oz	Founder-assisted workflow using current runtime and agents.	Will creators pay for a verified vertical slice before automation exists?	Five paid projects; at least three request a second iteration or continuation.
UX prototype	Clickable intent-plan-proof workspace with real or staged viewport.	Do users understand authority, constraints, cost, failure, and proof?	Users complete the flow without coaching and correctly predict rollback behavior.
Private alpha	Narrow live Studio with one archetype and two build targets.	Can the product create repeated verified iterations over four weeks?	Retention, accepted checkpoints, declining manual intervention, and repeat paid use.

Initial alpha thresholds—hypotheses, not forecasts

<60m

median time to first verified playable

≥60%

meaningful requests reach proof without manual code editing

≥3

accepted verified iterations per retained creator per week

≥35%

D7 creators continue the same project

Revise these thresholds after the concierge cohort. They exist to make failure legible, not manufacture success.

12. Measure accepted verified progress

North Star

Verified Playable Iterations per Weekly Active Creator (VPI/WAC)

A VPI is a prompt-derived checkpoint that runs, produces required evidence, passes agreed checks, and is accepted or continued by the creator rather than reverted.

This metric aligns the product with actual progress. Raw prompts reward chatter; generated lines reward bloat; build minutes reward inefficiency. VPI/WAC rewards usable creative throughput.

Objective	Metric	What it diagnoses
Activation	Time to First Verified Playable; percent reaching first checkpoint.	Onboarding and initial value.
Delegation quality	Verified completion, manual intervention, rollback/rework.	Whether autonomy removes or creates hidden labor.
Trust	Proof-card use, acceptance after failure, false-positive incidents.	Whether evidence and recovery are credible.
Retention	D7, W4, D30 same-project continuation; sessions per active project.	Whether Studio survives novelty.

Objective	Metric	What it diagnoses
Economics	Compute cost per accepted VPI, gross margin, top-up rate, failed-work credits.	Sustainability and fairness.
Platform	Native build success, crash-free sessions, device performance pass rate.	Portability promise.
Guardrails	Intent drift, provenance, security, and budget overruns.	Externalities growth metrics can hide.

Pricing architecture—test ranges

ThreeNative remains open source and free. Studio should use subscription + credits because value includes persistent product capabilities and variable model/compute cost.

Free / evaluation

\$0

Limited/public projects, local or runtime preview, small credit grant, framework, community builds.

Goal: first verified playable.

Creator hypothesis

\$20–39/month

Private projects, persistent intent/history, managed desktop/mobile builds, included work credits, top-ups.

Goal: recurring solo project.

Pro hypothesis

\$79–149/month

Parallel agents, more builds/ devices, collaboration, higher limits, priority execution, advanced proof.

Goal: serious creators and small teams.

Credit UX rules

- Estimate work class, range, and stop conditions before execution.
- Let users set task and monthly budget caps.
- Checkpoint useful partial progress before a budget is exhausted.
- Refund infrastructure failure and distinguish it from useful attempted work.
- Price by understandable work units and value tiers, not an opaque raw-token counter.
- Optimize internally for cost per accepted VPI.

Willingness-to-pay test: before final SaaS pricing, sell a founder-assisted verified vertical slice in a range such as **\$99–299**. Payment and a request for continued work are stronger evidence than waitlist signups.

PMF measurement

Use the Sean Ellis question only after a cohort has experienced the core loop at least twice recently: “How would you feel if you could no longer use Studio?” The widely cited benchmark is 40% “very disappointed,” but segmentation and observed retention matter more than treating 40% as a magic switch.^[19]

13. Supplemental Lean Canvas

Problem

1. AI-generated 3D games become fragile and disposable.
2. Creators cannot confidently verify long autonomous changes.
3. Native shipping requires platform expertise and wrapper compromises.

Customer segments

Beachhead: technical AI-first creators.

Adjacent: web developers, technical designers, small AI-native teams.

Anti: AAA/manual-control production.

Unique value proposition

Direct the game. ThreeNative builds, proves, and ships it.

Editable Three.js project, native runtime, reusable game systems, persistent intent, visible proof.

Solution

- Intent-plan-build-verify Studio.
- Open game framework.
- Agent inspection and playtesting.
- Managed native builds.

Channels

- GitHub/npm OSS adoption.
- Three.js and AI communities.
- Reference-to-game demos.
- Creator case studies and templates.
- Plugin authors later.

Revenue

- Studio subscription + credits.
- Managed builds and compute.
- Team/pro plans.
- Plugin take rate later.
- Game distribution later.

Cost structure

- Inference and compute.
- Native build infrastructure/devices.
- Storage, bandwidth, asset generation.
- Runtime/framework engineering.
- Support/moderation/payments later.

Key metrics

- VPI/WAC.
- TTFVP.
- D7/W4 continuation.
- Cost per accepted VPI.
- Build pass rate.
- Manual intervention and rollback.

Unfair advantage to build

A compounding game-evaluation system: verified task history, intent semantics, native conformance, reusable systems, plugin manifests, and an open Three.js ecosystem models already understand.

Riskiest assumptions

Assumption	Impact if false	Confidence	Cheapest test
Creators pay for verified autonomy, not just generation.	Fatal	Low	Paid concierge projects and repeat purchases.
The target returns for multi-session work.	Fatal	Low	Four-week cohort and same-project retention.

Assumption	Impact if false	Confidence	Cheapest test
Visible proof materially increases trust and acceptance.	High	Medium-low	A/B proof card versus ordinary chat completion.
One narrow game archetype is broad enough.	High	Medium-low	Recruit to a defined archetype; record rejection reasons.
Native performance/packaging changes purchasing behavior.	High	Medium	Message test and paid build add-on.
Subscription + credits feels predictable and fair.	Medium	Medium-low	Upfront estimates, mock invoices, and pricing interviews.
Open source drives adoption without erasing Studio value.	Medium	Medium	Track OSS-to-Studio activation and self-host objections.

14. Studio first; plugins second; player marketplace last

The plugin ecosystem and a “Steam for vibe-coded games” are compelling, but they are network-effect businesses layered on top of a creator tool. Launching them before creator retention multiplies cold-start, moderation, quality, and discovery problems.



A ThreeNative plugin must be agent-readable and verifiable

Contract	Required content
Semantic manifest	Capability such as “humanoid ragdoll,” “quest system,” or “voxel terrain”; when to use it; inputs, outputs, limitations, and examples.
Permission surface	Filesystem, network, native APIs, project mutation, runtime hooks, and telemetry.
Compatibility	ThreeNative/Three.js versions, runtime targets, GPU requirements, feature flags, and licensing.
Verification pack	Deterministic sample, conformance tests, performance budget, expected visual/behavior output.
Agent examples	Prompt-to-plan patterns, failure recovery, override points, and raw APIs.
Provenance	Author, source, license, generated-content metadata, and disclosures.

Launch plugins when

- ✓ Studio has retained projects.
- ✓ Repeated requests reveal stable extension seams.
- ✓ At least ten useful plugins have maintainers.
- ✓ Automated version/conformance tests exist.
- ✓ Creators already request third-party capabilities.

Do not launch when

- ✗ Core APIs change weekly.
- ✗ The store is intended to create demand rather than serve it.
- ✗ Quality review is manual and unscalable.
- ✗ Plugins cannot prove target-platform compatibility.
- ✗ There is no support or security model.

The future player marketplace

The long-term concept is coherent: creators build with Studio, publish inside the ThreeNative ecosystem, pay to list, and players purchase verified games. Existing platforms illustrate useful mechanics. Steam charges a \$100 per-product Direct fee that is recouped after \$1,000 in adjusted gross revenue, functioning partly as a spam bond.^[20] Asset stores show that curation and a take rate can support an extension economy.^[21]

Recommended sequence: begin with a curated showcase and one-click export to existing channels such as Steam, itch.io, web, and app stores. Add payments and discovery only after enough games pass a visible quality bar. Never make the internal marketplace a lock-in requirement.

Game-marketplace launch gates

- A meaningful supply of complete games, not generated scenes.
- Automated checks for boot, input, performance, malware, prohibited content, and asset provenance.
- Human curation, refunds, payments, taxation, moderation, support, ratings integrity, and discovery.
- Evidence that players return independently of creator acquisition.
- A clear answer to why a player chooses this catalog over Steam, itch.io, Roblox, mobile stores, or browser portals.

The prior PS4/PS5 report can remain a low-budget portability proof, but console work has no product-market priority until a paying creator or partner demands it.

15. Roadmap: buy evidence before complexity

Phase	Goal	Scope	Exit gate
0. Discovery 0–6 weeks	Confirm target, pain, delegation mindset, willingness to pay.	20–30 interviews; five paid concierge games; three message tests; workflow mapping.	Clear high-expectation segment, repeated pain, and at least three paid continuations or second projects.
1. Private alpha 6–12 weeks	Prove repeated verified playable iterations.	10–20 creators; one genre family; intent-plan-build-proof; one desktop and one mobile lane.	TTFVP/VPI thresholds, $\geq 35\%$ D7 continuation, falling intervention.
2. Closed paid beta 3–6 months	Prove retention and unit economics.	Subscription/credits, private projects, persistent memory, stronger primitives, reliability.	50–100 paying creators, gross-margin path, W4 retention, PMF signal by segment.
3. Product depth 6–12 months	Expand within the same customer before expanding the market.	More game patterns, platforms, collaboration, asset pipeline, verification library, plugin SDK beta.	Repeated third-party extension demand and stable APIs.
4. Ecosystem 12–24 months	Create supply-side network effects.	Curated plugins/assets, certified manifests, revenue share, creator showcase.	Healthy plugin usage, maintainers, compatibility automation, support economics.
5. Distribution later	Create player-side demand and game sales.	Curated catalog, discovery, payments, listing economics, player identity/community.	Enough verified games and independent player retention for a two-sided market.

First 90 days

WEEKS 1–3

Find the real pain

Recruit in Three.js, AI coding, indie, and web-dev communities. Interview the last attempted game. Collect abandoned repos and build stories.

WEEKS 2–6

Sell concierge outcomes

Deliver five vertical slices. Track founder time, agent failures, proof used, and requests for paid continuation.

WEEKS 5–8

Test the UX

Prototype intent board, plan, permissions, proof card, failure, rollback, and budget comprehension.

WEEKS 7–12

Ship the narrow alpha

10–20 creators, one archetype, instrument VPI/TTFVP/cost/intervention, run a four-week cohort.

Founder decision gates

Continue and invest when

✓ Users return to the same project across weeks.

✓ They delegate larger changes after seeing proof.

✓ They pay to continue rather than merely praise the demo.

✓ Verification catches real defects and changes acceptance.

✓ Framework primitives reduce cost and failure over time.

Pause, narrow, or pivot when

✗ Use is one-shot novelty.

✗ Creators require a full manual editor before continuing.

✗ Generic agents plus local Three.js are good enough.

✗ Proof cards are ignored or distrusted.

✗ Native build value does not influence conversion or retention.

Risk register

Risk	Mitigation
AI slop and aesthetic sameness	Reference-first direction, intent locks, curated styles, creator acceptance, no indiscriminate defaults.
False verification	Layered evidence, deterministic scenarios, cross-checks, explicit uncertainty, human final judgment.
Credit anxiety	Estimates, caps, checkpoints, infra refunds, cost-per-VPI optimization.
Incumbents copy AI workflows	Open/raw Three.js, AI-first UX, native runtime, low overhead, verified-task corpus, OSS distribution.
Open-source cloning	Monetize orchestration, compute, builds, project intelligence, collaboration, and ecosystem.
Marketplace cold start	Delay, curate, export to existing stores, enforce supply and quality gates.
IP and provenance	Asset metadata, source records, user controls, moderation, opt-in data use.
Platform reliability	Narrow supported lanes, conformance, hardware evidence, fail closed, visible support tiers.
Framework bloat	Extract from repeated Studio use, preserve raw Three.js, avoid mandatory architecture.

The next milestone is not “more platform support.” It is a cohort of creators who repeatedly direct, accept, pay for, and continue verified game changes.

16. Discovery interview guide

Recruit people who attempted an AI-assisted 3D game in the last six months. Prefer artifacts over opinions: ask for the repo, screenshots, build, chat history, abandoned branch, or platform error.

Core questions

1. Tell me about the last 3D game or interactive prototype you tried to make. What triggered it?
2. What tools did you start with, and why?
3. Show me where AI helped most. What did you delegate?
4. When did the project become harder to change or trust?
5. Describe the last regression AI introduced. How did you find it?
6. How did you decide whether a change was actually done?
7. What happened when you tried desktop or mobile packaging?
8. Which details must remain under direct control? Which implementation choices do you happily delegate?
9. What did you abandon, rewrite, or pay for?
10. What would make you continue this project for another month?

Avoid

- “Would you use ThreeNative?”
- “Do you like AI?”
- “Would a marketplace be cool?”
- Feature voting without evidence of past pain.
- Explaining the product before understanding the current workflow.

Moderated prototype test

1. Give the participant a blank Studio project and their own reference image.
2. Ask them to define the intent contract without coaching.
3. Present a plan with one deliberate questionable assumption; observe correction.
4. Ask them to choose permission and budget levels.
5. Simulate execution, failed playtest, automatic correction, and proof card.
6. Ask what changed, what they trust, what they would inspect, and whether they accept.
7. Offer source access and a raw Three.js override.
8. Ask for the next direction, then ask whether they would pay to continue.

Recommended product brief

Field	Decision
Vision	Make AI reliably capable of building real, editable, native 3D games while the creator retains direction, constraints, visibility, and override.
Target	AI-first technical creators who value outcome over manual authoring and have felt generated-game brittleness.

Field	Decision
Problem	AI rapidly generates Three.js demos, but projects often fail to remain trustworthy, maintainable, performant, and shippable.
Promise	Direct the game. ThreeNative builds, proves, and ships it.
Workflow	Reference/brief → intent → plan → permission/budget → build → self-play/inspect → proof → accept/refine/revert → export.
Framework	Open-source batteries, native runtime, agent-facing inspection/verification, and upstream Three.js escape hatch.
Studio	Commercial orchestration, project intelligence, managed agents, compute, verification, builds, collaboration.
North Star	Verified Playable Iterations per Weekly Active Creator.
Non-goals	Full manual editor, Unity parity, AAA, mandatory ECS, arbitrary genre breadth, marketplace-first, console-first.
90-day proof	Five paid concierge projects; 10–20 alpha users; repeated same-project use; measurable verified iterations; two reliable targets.

// ThreeNative turns the Three.js programming model that AI already understands into an open native game framework, then sells the agentic Studio that can build, play, verify, and ship games on top of it.

Research references

1. **Dan Olsen / Wiley.** *The Lean Product Playbook* and six-step Lean Product Process.
<https://onlinelibrary.wiley.com/doi/book/10.1002/9781119154822>
2. **Dan Olsen / Wiley.** Product-Market Fit Pyramid; conclusion; problem space versus solution space.
<https://onlinelibrary.wiley.com/doi/abs/10.1002/9781119154822.ch15>
<https://onlinelibrary.wiley.com/doi/10.1002/9781119154822.ch2>
3. **Dan Olsen / Wiley.** End-to-end case study and testing design artifacts before coding.
<https://onlinelibrary.wiley.com/doi/10.1002/9781119154822.ch11>
4. **Leanstack.** Lean Canvas resources.
<https://leanstack.com/lean-canvas>
5. **Stack Overflow.** 2025 Developer Survey, AI section.
<https://survey.stackoverflow.co/2025/ai>
6. **Stack Overflow.** 2025 survey methodology.
<https://survey.stackoverflow.co/2025/methodology>
7. **Roblox.** "Roblox Studio is Going Agentic," 15 April 2026.
<https://about.roblox.com/newsroom/2026/04/roblox-studio-going-agentic>
8. **Unity.** In-editor AI Assistant: Ask, Plan, Agent, permissions, reversibility, verification.
<https://unity.com/blog/unity-ai-assistant-ask-plan-agent-mode-explained>
9. **Google Cloud / Harris Poll.** 2025 Games Report.
<https://cloud.google.com/resources/games-report>
10. **npm.** Three.js package description, downloads, and dependents snapshot.
<https://www.npmjs.com/package/three>
11. **Rosebud AI.** AI game creator capabilities.
<https://rosebud.ai/ai-game-creator>
12. **GDevelop.** Open-source engine, extensions, JavaScript, AI, and exports.
<https://gdevelop.io/game-makers>
13. **Dan Olsen / Wiley.** Value proposition and Kano classification.
<https://onlinelibrary.wiley.com/doi/10.1002/9781119154822.ch5>
14. **Microsoft HAX Toolkit / Microsoft Research.** Guidelines for Human-AI Interaction.
<https://www.microsoft.com/en-us/haxtoolkit/ai-guidelines/>
<https://www.microsoft.com/en-us/research/publication/guidelines-for-human-ai-interaction/>
15. **Google PAIR.** People + AI Guidebook and Feedback + Control.
<https://pair.withgoogle.com/old-gb/>
<https://pair.withgoogle.com/guidebook-v2/chapter/feedback-controls/>
16. **Dan Olsen / Wiley.** Specify the MVP feature set.
<https://onlinelibrary.wiley.com/doi/10.1002/9781119154822.ch6>
17. **Dan Olsen / Wiley.** Create the MVP prototype; concierge and Wizard of Oz tests.
<https://onlinelibrary.wiley.com/doi/10.1002/9781119154822.ch7>
18. **Dan Olsen / Wiley.** Test the MVP with customers.
<https://onlinelibrary.wiley.com/doi/10.1002/9781119154822.ch9>
19. **First Round Review.** Superhuman's application of the Sean Ellis PMF survey.
<https://review.firstround.com/how-superhuman-built-an-engine-to-find-product-market-fit/>
20. **Valve Steamworks.** Steam Direct fee.
<https://partner.steamgames.com/steamdirect/>

21. **Unity Asset Store.** Publisher submission and revenue economics.

<https://assetstore.unity.com/publishing/publish-and-sell-assets>

Founder inputs. Product architecture, category corrections, the open-source/commercial split, delegation mindset, platform thesis, escape-hatch philosophy, plugin ecosystem, and future game marketplace were synthesized from the product conversation commissioning this report. They are treated as founder intent, not independently verified market facts.